

쿠버네티스

그리고 약간의 Slopsquatting

이현지

쿠버네티스(K8s)란?

쿠버네티스(Kubernetes)는 컨테이너화된 애플리케이션의 배포, 확장, 관리를 **자동으로** 처리해주는 오픈소스 플랫폼입니다. 줄여서 K8s라고도 부르며, 그리스어로 '조타수'를 뜻합니다. 배의 방향을 잡고 항해를 이끄는 사람처럼 쿠버네티스는 복잡한 컨테이너 환경의 방향을 잡아주는 역할을 합니다.



쿠버네티스 문서

쿠버네티스는 컨테이너화된 애플리케이션의 배포, 확장 및 관리를 자동화하기 위한 오픈소스 컨테이너 오케스트레이션 엔진이다. 본 오픈소스 프로젝트는 Cloud Native Computing Foundation이 주관한다.



쿠버네티스가 왜 필요하고 무엇을 할 수 있나

컨테이너는 애플리케이션을 포장하고 실행하는 좋은 방법이다. 프로덕션 환경에서는 애플리케이션을 실행하는 컨테이너를 관리하고 가동 중지 시간이 없는지 확인해야 한다. 예를 들어 컨테이너가 다운되면 다른 컨테이너를 다시 시작해야 한다. 이 문제를 시스템에 의해 처리한다면 더 쉽지 않을까?

그것이 쿠버네티스가 필요한 이유이다! 쿠버네티스는 분산 시스템을 탄력적으로 실행하기 위한 프레임 워크를 제공한다. 애플리케이션의 확장과 장애 조치를 처리하고, 배포 패턴 등을 제공한다. 예를 들어, 쿠버네티스는 시스템의 카나리아 배포를 쉽게 관리할 수 있다.

쿠버네티스는 다음을 제공한다.

보안관점에서 보면 좋을 것 같은 부분 TOP3.

- **시크릿과 구성 관리** 쿠버네티스를 사용하면 암호, OAuth 토큰 및 SSH 키와 같은 중요한 정보를 저장하고 관리할 수 있다. 컨테이너 이미지를 재구성하지 않고 스택 구성에 시크릿을 노출하지 않고도 시크릿 및 애플리케이션 구성을 배포 및 업데이트할 수 있다.

GitHub 토큰

- 컨테이너 레지스트리 인증정보
- CI/CD 배포 토큰
- 클라우드 접근 키
- 이미지 서명 키
- Kubernetes 서비스 계정 토큰

보안관점에서 보면 좋을 것 같은 부분 TOP3.

- **자동화된 롤아웃과 롤백** 쿠버네티스를 사용하여 배포된 컨테이너의 원하는 상태를 서술할 수 있으며 현재 상태를 원하는 상태로 설정한 속도에 따라 변경할 수 있다. 예를 들어 쿠버네티스를 자동화해서 배포용 새 컨테이너를 만들고, 기존 컨테이너를 제거하고, 모든 리소스를 새 컨테이너에 적용할 수 있다.

새 이미지 생성

→ 이미지 저장소 등록

→ Deployment 수정

→ 새 파드 배포

→ 기존 파드 제거

보안관점에서 보면 좋을 것 같은 부분 TOP3.

- **자동화된 복구(self-healing)** 쿠버네티스는 실패한 컨테이너를 다시 시작하고, 컨테이너를 교체하며, '사용자 정의 상태 검사'에 응답하지 않는 컨테이너를 죽이고, 서비스 준비가 끝날 때까지 그러한 과정을 클라이언트에 보여주지 않는다.

악성 이미지가 배포됨

→ 악성 컨테이너가 실행됨

→ 컨테이너가 오류로 종료되더라도

→ Kubernetes는 악성 여부를 모르기 때문에 다시 실행할 수 있음

그럼 어디를 중점으로 봐야할까?

 Search this site

▸ Documentation

▸ Getting started

▾ Concepts

▸ Overview

▸ Cluster
Architecture

▾ Containers

Images

Container

Environment

Runtime Class

Container
Lifecycle HooksContainer
Runtime

Interface (CRI)

▸ Workloads

▸ Services, Load
Balancing, and
Networking

▸ Storage

▸ Configuration

▸ Security

Images

A container image represents binary data that encapsulates an application and all its software dependencies. Container images are executable software bundles that can run standalone and that make very well-defined assumptions about their runtime environment.

You typically create a container image of your application and push it to a registry before referring to it in a Pod.

This page provides an outline of the container image concept.

Note:

If you are looking for the container images for a Kubernetes release (such as v1.36, the latest minor release), visit [Download Kubernetes](#).

Image names

Container images are usually given a name such as `pause`, `example/mycontainer`, or `kube-apiserver`. Images can also include a registry hostname; for example: `fictional.registry.example/imagename`, and possibly a port number as well; for example: `fictional.registry.example:10443/imagename`.

If you don't specify a registry hostname, Kubernetes assumes that you mean the [Docker public registry](#). You can change this behavior by setting a default image registry in the [container runtime configuration](#).

After the image name part you can add a *tag* or *digest* (in the same way you would when using with commands like `docker` or `podman`). Tags let you identify different versions of the same series of

[✎ Edit this page](#)[✎ Create child page](#)[🔗 Create an issue](#)[🖨 Print entire section](#)

Image names

Updating images

Image pull policy

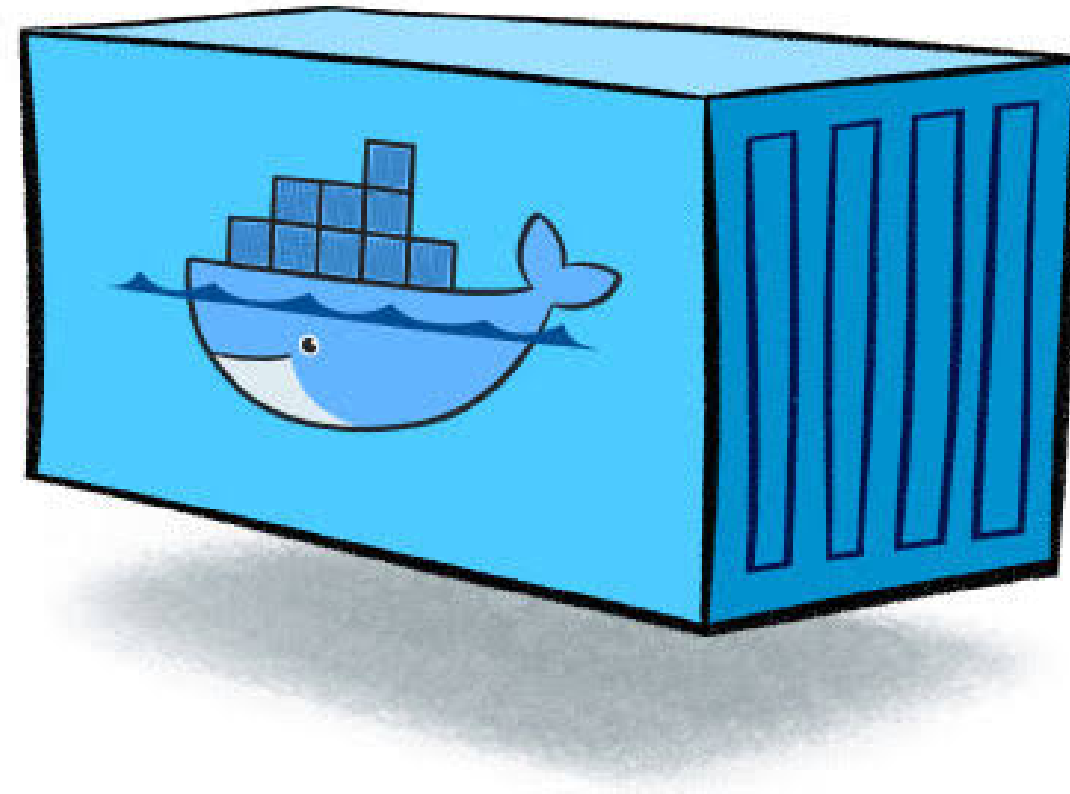
ImagePullBackOff

Image pull per runtime
classSerial and parallel image
pullsMaximum parallel image
pullsMulti-architecture images
with image indexes

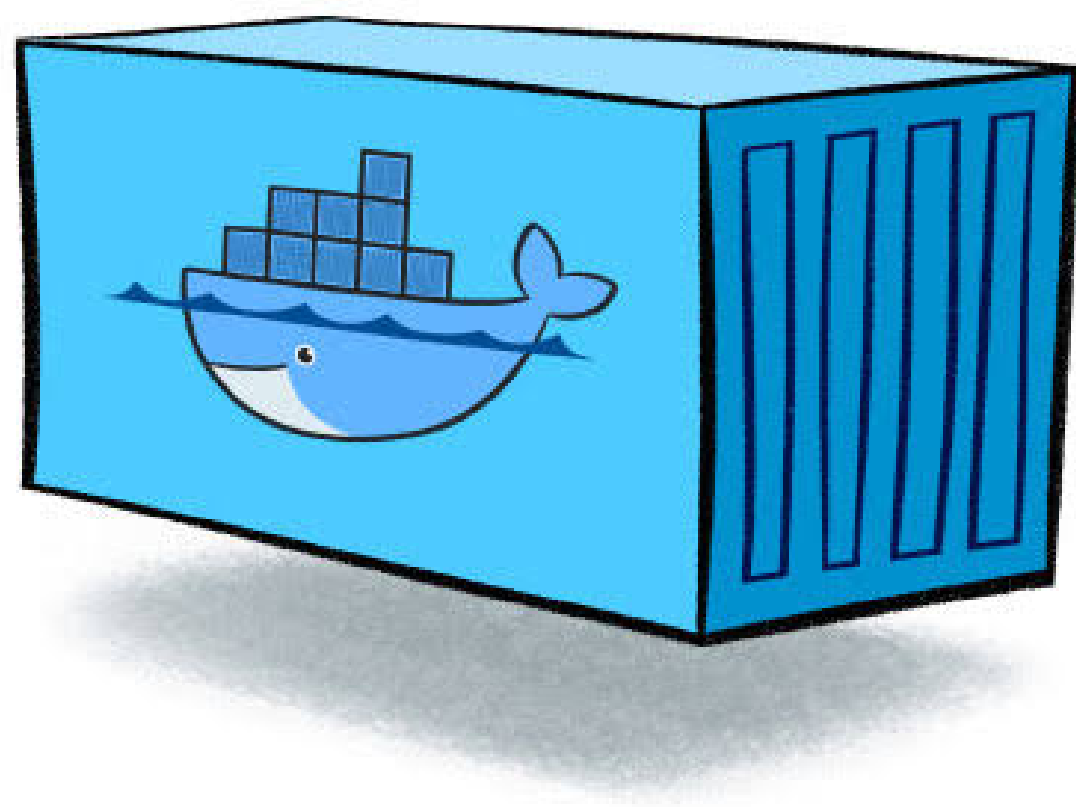
Using a private registry

Specifying `imagePullSecrets`
on a PodConfiguring nodes to
authenticate to a private
registryKubelet credential provider
for authenticated image
pulls

Interpretation of config.json



쿠버네티스는 애플리케이션을 실행할 때
먼저 **컨테이너 이미지**를 가져온다



애플리케이션 코드
+ 외부 라이브러리
+ 실행에 필요한 파일
+ 운영체제 구성요소

image: registry.example.com/team/my-app:v1.2.0

registry.example.com / team / my-app : v1.2.0
└─ 이미지 저장소 └─ 이미지 이름 └─ 태그

보안상 주의점

이미지 저장소 주소를 생략하면 의도하지 않은 공개 저장소의 이미지를 받을 수 있습니다.

yaml

출처가 명확하지 않음

image: my-app:v1

출처가 명확함

image: registry.company.example/team/my-app:v1

공급망 보안에서는 이미지 이름뿐 아니라 “어느 저장소에서 가져오는가?”도 중요합니다.



**이미지가 변조되거나 취약하면
쿠버네티스도 그대로 문제가 있는 프로그램을 실행**

태그 vs 다이제스트

태그

yaml

```
image: example/app:v1
```

태그는 이미지에 붙인 이름표입니다.

문제는 `v1` 이라는 이름표를 다른 이미지로 옮길 수 있다는 것입니다.

○ 채팅에 추가

오늘의 app:v1 → 정상 이미지 A

내일의 app:v1 → 변경되거나 변조된 이미지 B

배포 파일에는 계속 `app:v1` 이라고 적혀 있지만, 실제로 실행되는 내용은 달라질 수 있습니다.

특히 다음 방식은 피하는 것이 좋습니다.

yaml

```
image: example/app:latest
```

`latest` 는 보통 "가장 최근에 등록된 이미지"를 가리키므로, 나중에 다른 이미지로 바뀔 가능성이 큼니다.

Digest

yaml

```
image: example/app@sha256:45b23d...
```

digest는 이미지 내용으로 계산한 고유한 지문입니다.

이미지 파일이 조금이라도 달라지면 digest도 달라집니다.

○ 채팅에 추가

정상 이미지 A → sha256:111...

변조된 이미지 B → sha256:999...

따라서 digest를 지정하면 쿠버네티스는 정확히 그 이미지 하나만 실행합니다.

For example: 예를 들어:

```
cat <<EOF > pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: foo
  namespace: awesomeapps
spec:
  containers:
    - name: foo
      image: janedoe/awesomeapp:v1
      imagePullSecrets:
        - name: myregistrykey
EOF

cat <<EOF >> ./kustomization.yaml
resources:
- pod.yaml
EOF
```

This needs to be done for each Pod that is using a private registry.

비공개 레지스트리를 사용하는 각 포드에 대해 이 작업을 수행해야 합니다.

However, you can automate this process by specifying the `imagePullSecrets` section in a [ServiceAccount](#) resource. See [Add ImagePullSecrets to a Service Account](#) for detailed instructions.

Pod 실행 요청



registry-login Secret에서 인증정보 확인



Private Registry에 로그인



my-app 이미지 다운로드



컨테이너 실행

imagePullSecret이란?

Private Registry에서 이미지를 받으려면 아이디·비밀번호 또는 토큰이 필요합니다. 그 인증정보를 Kubernetes Secret에 보관한 것이 imagePullSecret입니다.

Docker config Secrets

If you are creating a Secret to store credentials for accessing a container image registry, you must use one of the following `type` values for that Secret:

```
{
  "auths": {
    "my-registry.example:5000": {
      "username": "tiger",
      "password": "pass1234",
      "email": "tiger@acme.example",
      "auth": "dGlnZXI6cGFzczEyMzQ="
    }
  }
}
```

추가로 중요한 점은 Secret 직접 조회 권한이 없어도, 해당 네임스페이스에 Pod나 Deployment를 생성할 수 있다면 Secret을 Pod에 연결하여 간접적으로 읽을 수 있다는 것

Caution: 주의:

The `auth` value there is base64 encoded; it is obscured but not secret. Anyone who can read that Secret can learn the registry access bearer token.

인증 값은 base64로 인코딩되어 있습니다. 그것은 가려져 있지만 비밀은 아닙니다. 해당 비밀을 읽을 수 있는 사람은 누구나 레지스트리 액세스 전달자 토큰을 배울 수 있습니다.

It is suggested to use [credential providers](#) to dynamically and securely provide pull secrets on-demand.

필요에 따라 풀 비밀을 동적으로 안전하게 제공하려면 자격 증명 공급자를 사용하는 것이 좋습니다.

**imagePullSecret의 인증정보는 Base64로 표현될 뿐 암호화된 것이 아닙니다.
또한 Kubernetes Secret은 별도 설정을 하지 않으면 etcd에 암호화되지 않은 상태로 저장됩니다.**

SO?

imagePullSecret을 읽을 수 있는 사람은

그 안의 아이디·비밀번호 또는 토큰을 알아내 **Private Registry**에 접근할 수 있다

값이 Base64로 표시되지만, Base64는 암호화가 아니라 단순 변환이므로 누구나 다시 원래 값으로 바꿀 수 있습니다.

imagePullSecret 에는 이런 정보가 들어 있습니다.

○ 채팅에 추가

Registry 주소

아이디

비밀번호 또는 접근 토큰

따라서 다음 권한을 아무에게나 주면 안 됩니다.

yaml



```
resources: ["secrets"]  
verbs: ["get", "list", "watch"]
```

- `get` : 특정 Secret을 읽을 수 있음
- `list` : Secret 목록과 내용을 가져올 수 있음
- `watch` : Secret 생성·변경 내용을 계속 볼 수 있음

imagePullSecret의 읽기 권한과, 그 안에 저장된 Registry 계정의 권한을 모두 최소화하는 것이 결론

1. Kubernetes Secret 읽기 권한 최소화

“누가 imagePullSecret 안의 Registry 인증정보를 볼 수 있는가?”를 제한

2. Registry 계정 권한 최소화

Secret이 실수나 공격으로 노출되더라도 할 수 있는 행동을 제한



Kubernetes API

Server Bypass

Risks

Linux kernel

security

constraints for

Pods and

containers

**Security
Checklist**

Application

Security

Checklist

► Policies

► Scheduling,
Preemption and
Eviction► Cluster
Administration► Windows in
Kubernetes► Extending
Kubernetes

Images

- ☐ Minimize unnecessary content in container images.
- ☐ Container images are configured to be run as unprivileged user.
- ☐ References to container images are made by sha256 digests (rather than tags) or the provenance of the image is validated by verifying the image's digital signature at deploy time [via admission control](#).
- ☐ Container images are regularly scanned during creation and in deployment, and known vulnerable software is patched.

Container image should contain the bare minimum to run the program they package. Preferably, only the program and its dependencies, building the image from the minimal possible base. In particular, image used in production should not contain shells or debugging utilities, as an [ephemeral debug container](#) can be used for troubleshooting.

Build images to directly start with an unprivileged user by using the [USER instruction in Dockerfile](#). The [Security Context](#) allows a container image to be started with a specific user and group with `runAsUser` and `runAsGroup`, even if not specified in the image manifest. However, the file permissions in the image layers might make it impossible to just start the process with a new unprivileged user without image modification.

Avoid using image tags to reference an image, especially the `latest` tag, the image behind a tag can be easily modified in a registry. Prefer using the complete `sha256` digest which is unique to the image manifest. This policy can be enforced via an [ImagePolicyWebhook](#). Image signatures can also be automatically [verified with an admission controller](#) at deploy time to validate their authenticity and integrity.

Scanning a container image can prevent critical vulnerabilities from being deployed to the cluster

[✎ Edit this page](#)[✎ Create child page](#)[🔔 Create an issue](#)[🖨 Print entire section](#)

Authentication & Authorization

Network security

Pod security

Enabling Seccomp

Enabling AppArmor or
SELinux

Logs and auditing

Pod placement

Secrets

Images

Admission controllers

What's next

Container image should contain the bare minimum to run the program they package. Preferably, only the program and its dependencies, building the image from the minimal possible base. In particular, image used in production should not contain shells or debugging utilities, as an ephemeral debug container can be used for troubleshooting.

- 실행에 피리요한 파일만 있어야지 불필요한 도구까지 넣으면 공격자 악용 가능성 0
- 이미지 안에 curl과 bash가 있다면 공격자는 추가 악성코드를 다운로드하고 실행하기 EZ

```
yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
```

역할은 다음과 같습니다.

- `runAsNonRoot: true` : root 실행 금지
- `runAsUser: 10001` : 지정한 일반 사용자 ID로 실행
- `allowPrivilegeEscalation: false` : 실행 중 더 높은 권한을 얻지 못하게 제한

>> 이런식으로 강제 가능함

Build images to directly start with an unprivileged user by using the USER instruction in Dockerfile. The Security Context allows a container image to be started with a specific user and group with `runAsUser` and `runAsGroup`, even if not specified in the image manifest. However, the file permissions in the image layers might make it impossible to just start the process with a new unprivileged user without image modification.

- root가 아닌 사용자로
- 이미지 생성시 일반 사용자 생성 하고 해당 사용자로 실행

Avoid using image tags to reference an image, especially the `latest` tag,
the image behind a tag can be easily modified in a registry. Prefer using the
complete `sha256` digest which is unique to the image manifest. This policy
can be enforced via an [ImagePolicyWebhook](#). Image signatures can also be
automatically [verified with an admission controller](#) at deploy time to
validate their [authenticity and integrity](#).

- 태그 말고 digest 사용하자
- 하지만 악성이미지나 취약한 이미지도 고유한 digest를 가짐
- 그래서 이미지 서명 검증이 필요하다

- 이미지 취약점을 계속 검사하라
- 이미지 스캔의 목적은 가능한 취약점
과 예방에 대한 정보를 얻는 것
- 빌드 > 검사 > 업로드 > 배포전 검사
> 운영중정기검사

Scanning a container image can prevent critical vulnerabilities from being
deployed to the cluster alongside the container image. Image scanning
should be completed before deploying a container image to a cluster and is
usually done as part of the deployment process in a CI/CD pipeline. The
purpose of an image scan is to obtain information about possible
vulnerabilities and their prevention in the container image, such as a
[Common Vulnerability Scoring System \(CVSS\)](#) score. If the result of the
image scans is combined with the pipeline compliance rules, only properly
patched container images will end up in Production.

▶ 문서

▶ 시작하기

▼ 개념

▶ 쿠버네티스란 무엇인가?

▶ 클러스터 아키텍처

▶ 컨테이너

▶ 쿠버네티스에서의 윈도우

▶ 워크로드

▶ 서비스, 로드밸런싱, 네트워킹

▶ 스토리지

▶ 구성

▼ 보안

클라우드 네이티브 보안 개요

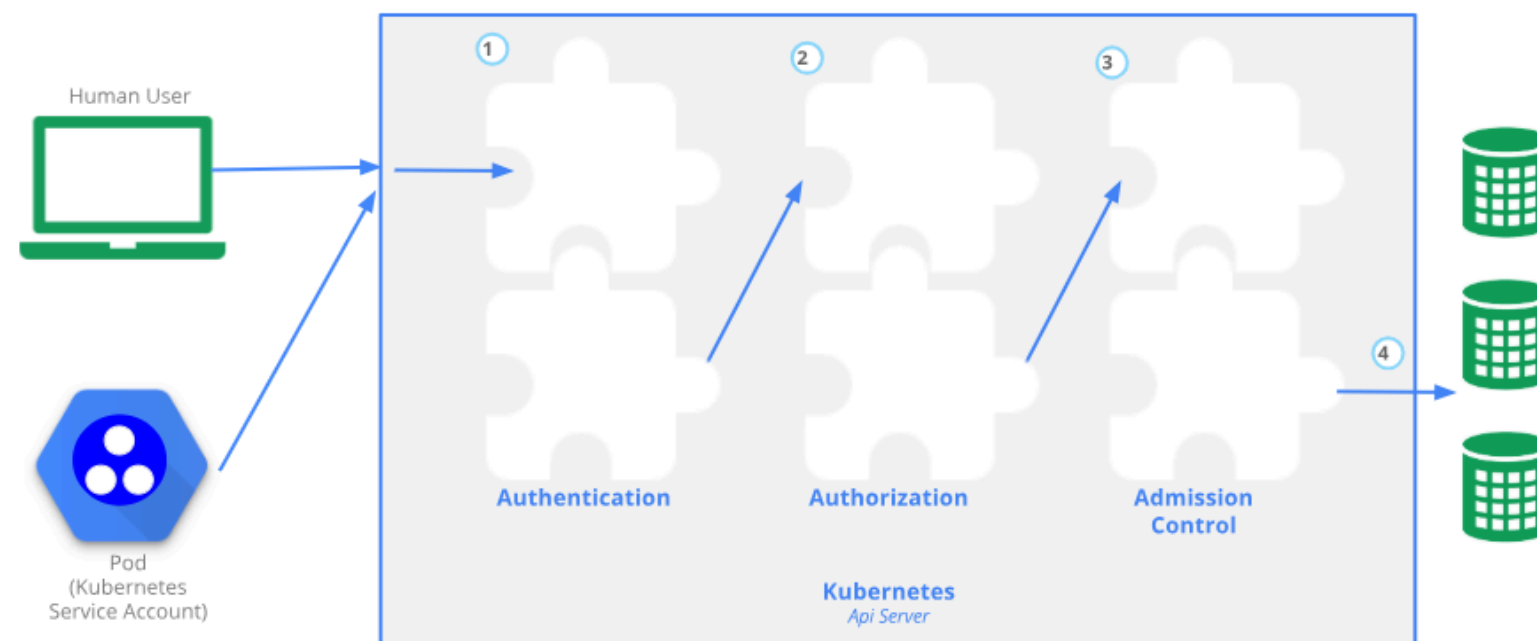
클라우드 네이티브 보안

파드 시큐리티 스

쿠버네티스 API 접근 제어하기

이 페이지는 쿠버네티스 API에 대한 접근 제어의 개요를 제공한다.

사용자는 `kubectl`, 클라이언트 라이브러리 또는 REST 요청을 통해 [API에 접근한다](#). 사용자와 쿠버네티스 서비스 어카운트 모두 API에 접근할 수 있다. 요청이 API에 도달하면, 다음 다이어그램에 설명된 몇 가지 단계를 거친다.


[페이지 편집](#)
[하부 페이지 생성](#)
[이슈 생성](#)
[전체 섹션 프린트](#)

전송 보안

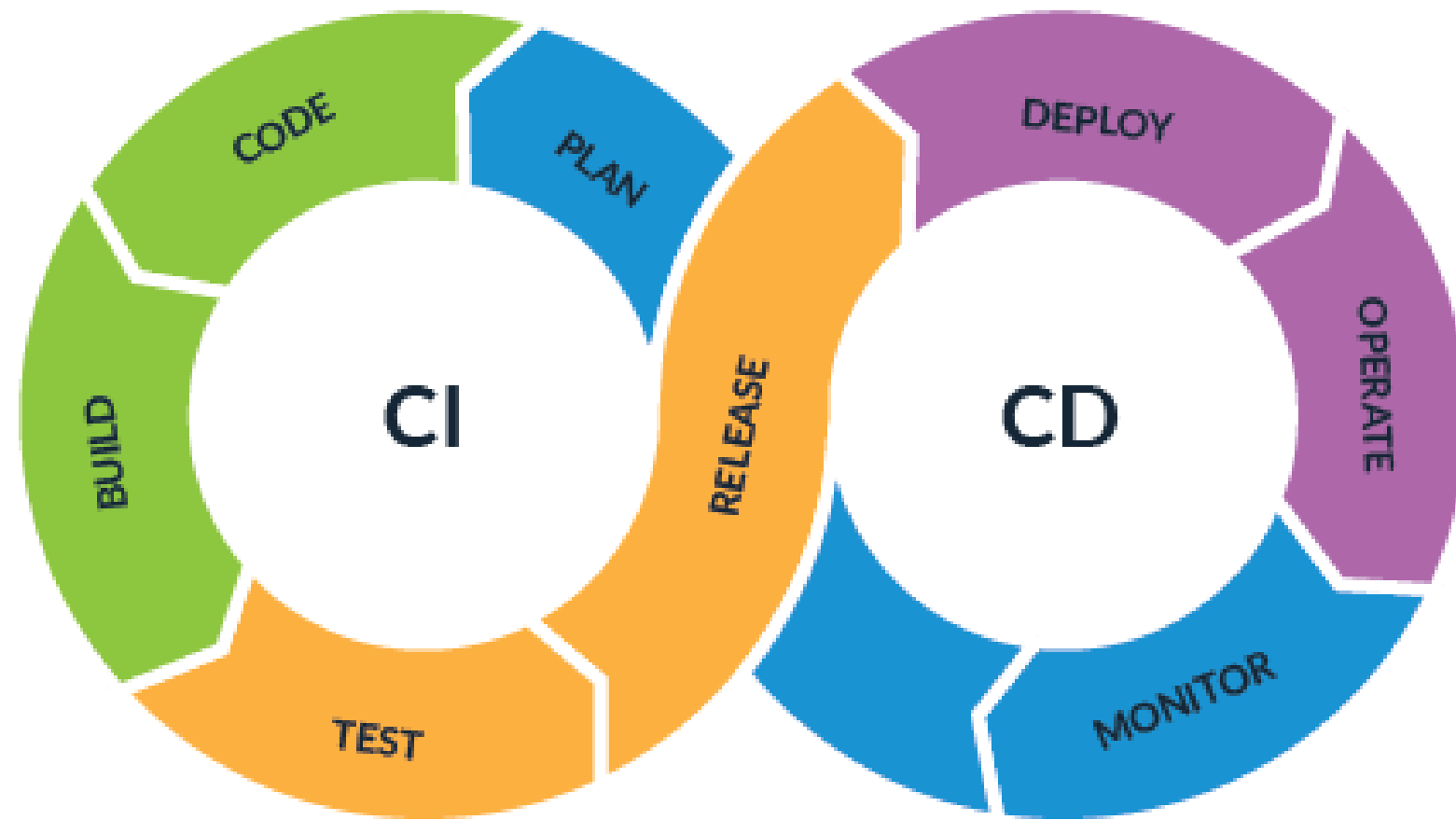
인증

인가

어드미션 제어

감사(Auditing)

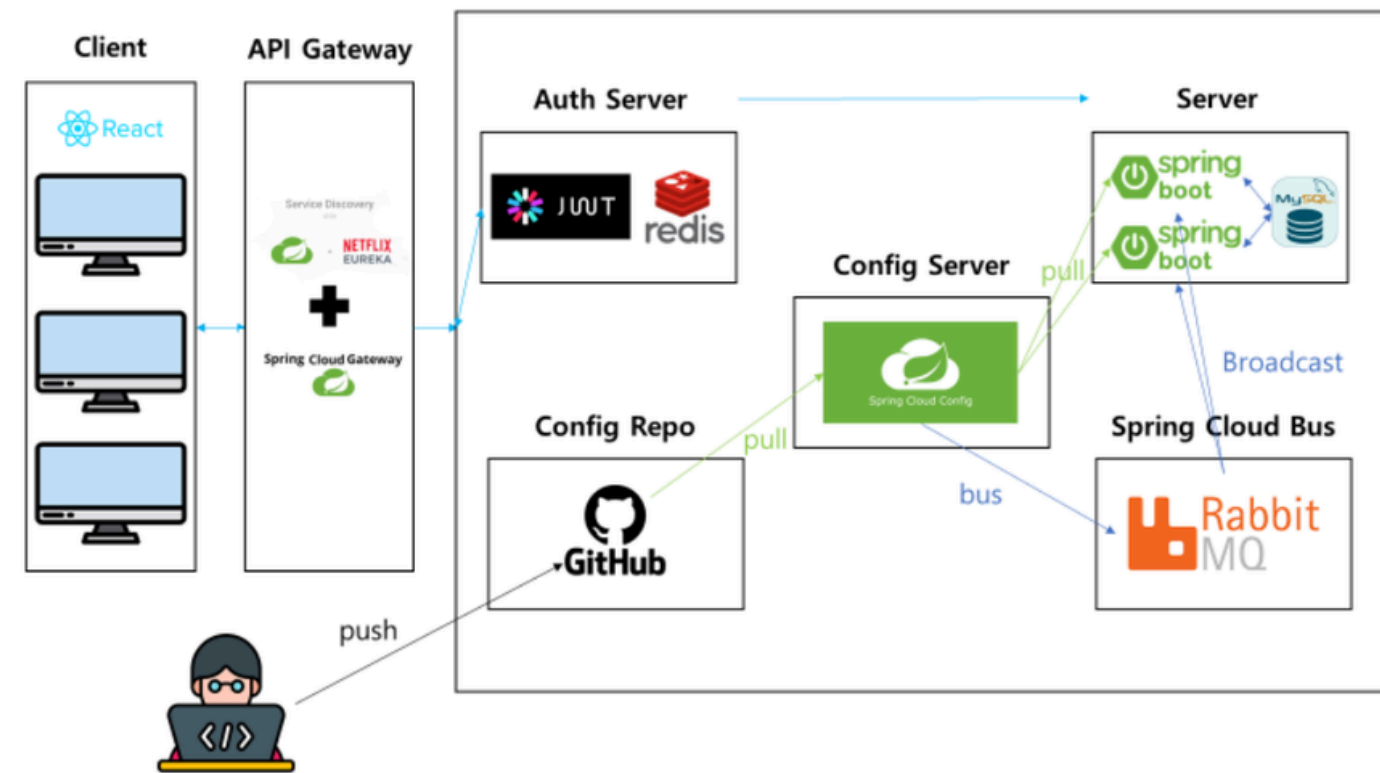
다음 내용



예를 들어 CI/CD가 애플리케이션 배포를 요청했다고 생각해보겠습니다.

1. 전송 보안(TLS)

설계



CI/CD — 암호화된 통신 —> Kubernetes API Server

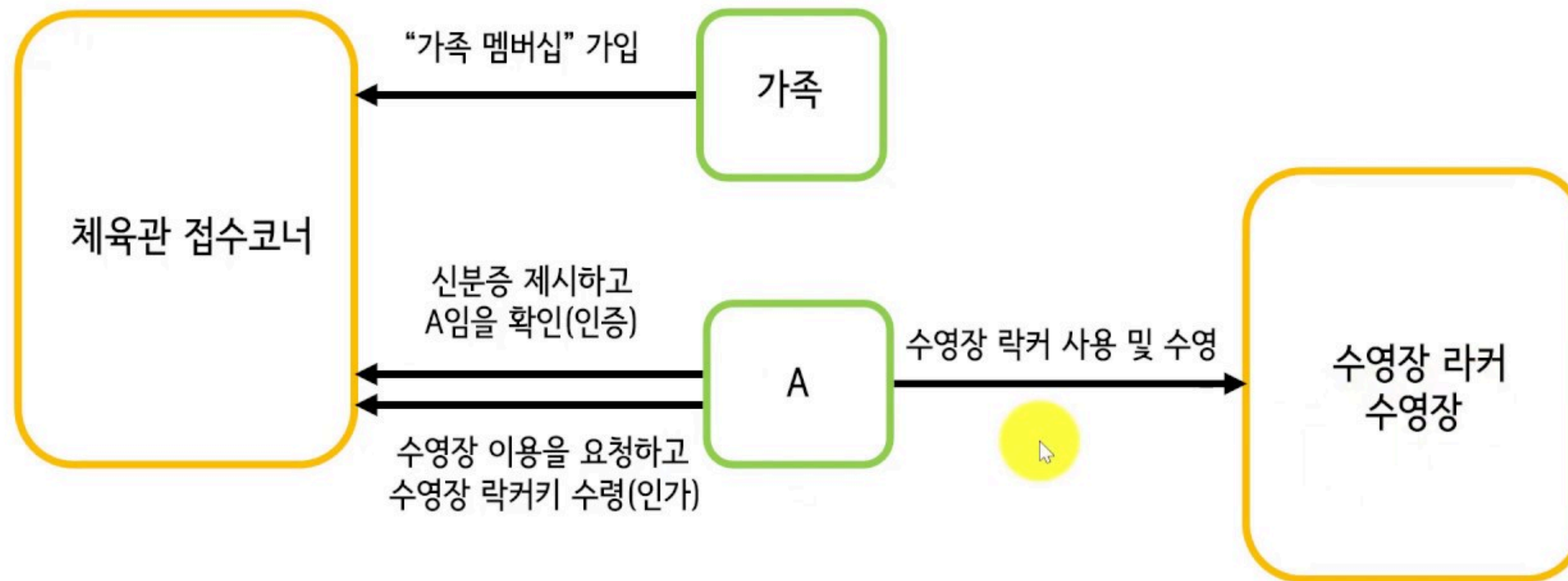
배포 정보와 토큰을 누군가 중간에서 훔쳐보거나 변조하지 못하도록 **TLS로 통신을 보호합니다.**

1. 전송 보안(TLS)

만약 클러스터가 사설 인증 기관을 사용한다면, 해당 CA 인증서를 복사하여 클라이언트의 `~/.kube/config` 안에 구성함으로써 연결을 신뢰하고 누군가 중간에 가로채지 않았음을 보장해야 한다.

진짜 API 서버와 안전하게 통신하고 있는가?

2. 인증(Authentication)



CI/CD가 제출한 인증서나 토큰을 확인

2. 인증(Authentication)

인증 단계로 들어가는 것은 온전한 HTTP 요청이지만 일반적으로 헤더 그리고/또는 클라이언트 인증서를 검사한다.

요청을 보낸 주체가 누구인가?

2. 인증(Authentication)

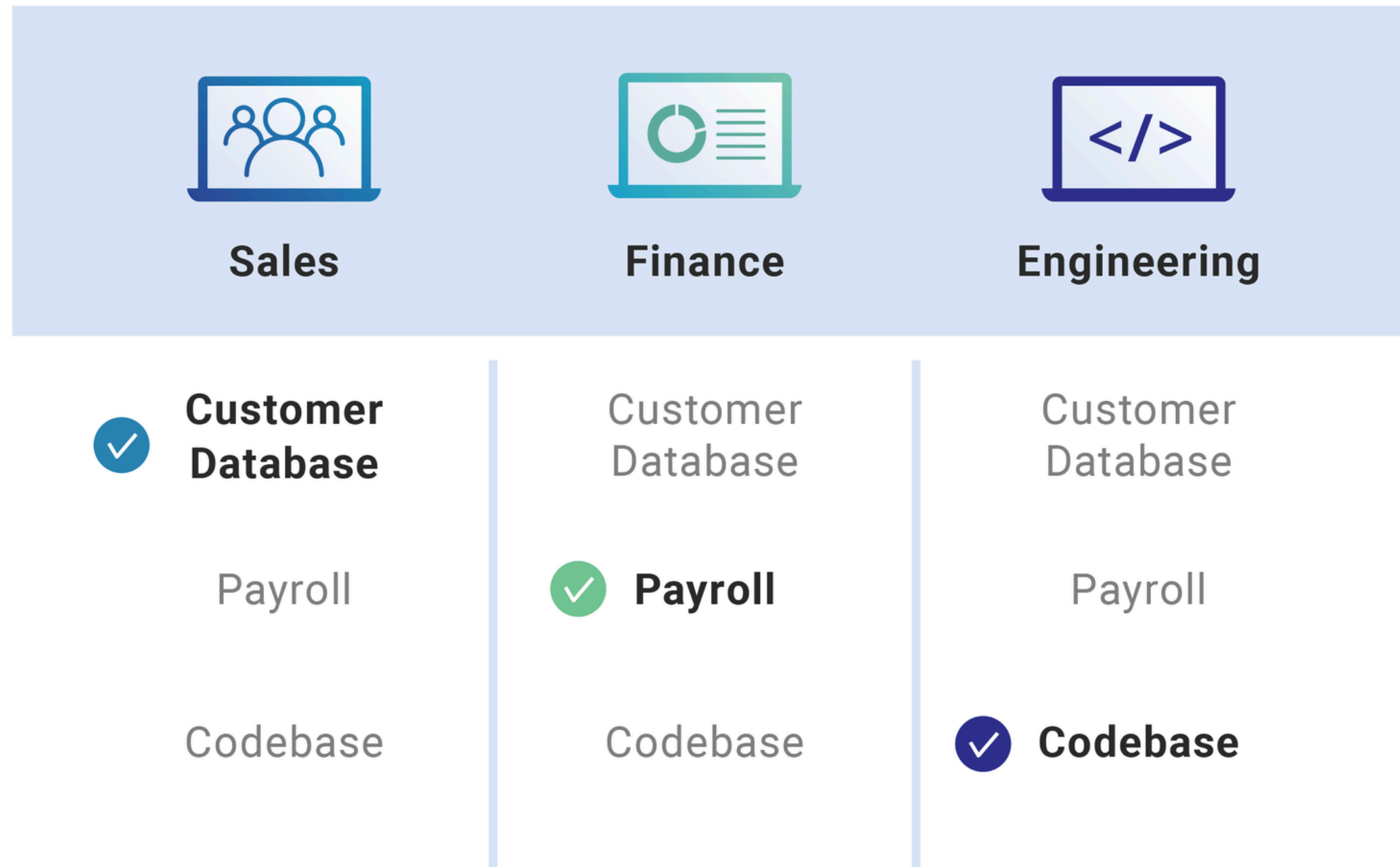
사용자 토큰 → developer로 확인

ServiceAccount 토큰 → ci-deployer로 확인

토큰이 잘못됨 → 401 거부

401 Unauthorized는 “누구인지 확인할 수 없다”는 의미입니다.

3. 인가(Authorization)



인증된 주체에게 요청한 작업을 수행할 권한이 있는지 확인
일반적으로 RBAC(역할기반접근제어)이 담당

3. 인가(Authorization)

요청은 요청자의 username, 요청된 작업 및 해당 작업이 영향을 주는 오브젝트를 포함해야 한다. 기존 정책이 요청된 작업을 완료할 수 있는 권한이 해당 사용자에게 있다고 선언하는 경우 요청이 인가된다.

이 계정이 이 리소스에 이 작업을 해도 되는가?

누가(subject)
+ 어디에서(namespace)
+ 무엇을(resource)
+ 어떤 행동으로(verb)

3. 인가(Authorization)

ci-deployer에게 다음 권한만 줬다고 가정해봅시다.

```
{
  "apiVersion": "abac.authorization.kubernetes.io/v1beta1",
  "kind": "Policy",
  "spec": {
    "user": "bob",
    "namespace": "projectCaribou",
    "resource": "pods",
    "readonly": true
  }
}
```

Bob이 다음과 같은 요청을 하면 'projectCaribou' 네임스페이스의 오브젝트를 읽을 수 있기 때문에 요청이 인가된다.

namespace: 작업할 공간이 projectCaribou

verb: get, 즉 조회

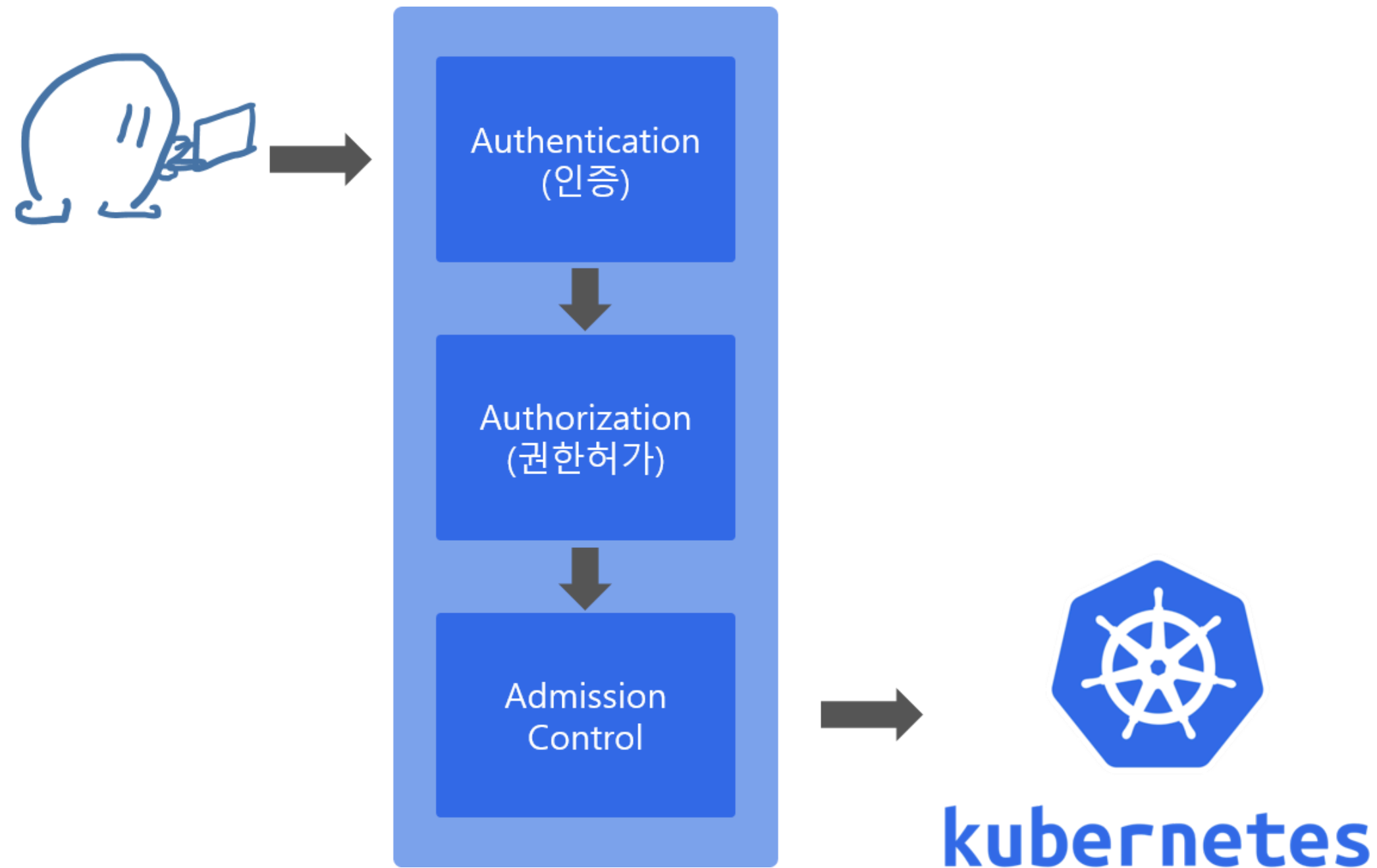
resource: 조회 대상은 Pod

3. 인가(Authorization)

projectCaribou의 Pod 조회(get) → 허용
projectCaribou의 Pod 생성(create) → 거부
projectCaribou의 Pod 수정(update) → 거부
projectFish의 Pod 조회(get) → 거부
projectCaribou의 Secret 조회(get) → 거부

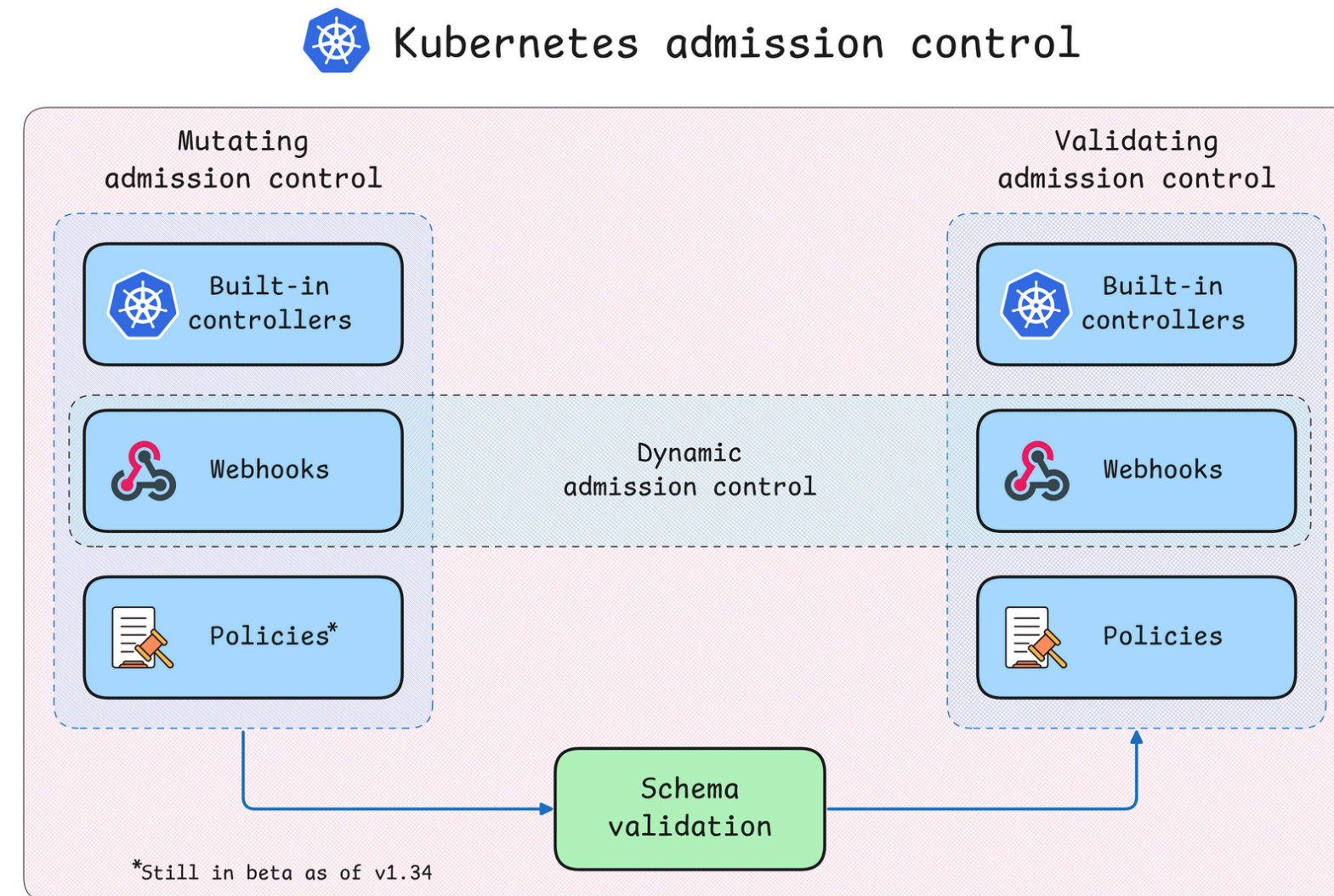
권한이 없으면 403 Forbidden으로 거부됩니다.

4. Admission Control



계정에 배포 권한이 있더라도 배포하려는 구체적인 내용이 안전한지 검사

4. Admission Control



이 배포 요청이 조직의 보안정책을 만족하는가?

4. Admission Control

어드미션 제어 모듈은 요청을 수정하거나 거부할 수 있는 소프트웨어 모듈이다.

인증 및 인가 모듈과 달리, 어드미션 제어 모듈이 거부되면 요청은 즉시 거부된다.

조직의 보안정책을 위반한 배포 요청을 거부할 수 있다는 뜻

4. Admission Control

어드미션 제어 모듈은 요청을 수정하거나 거부할 수 있는 소프트웨어 모듈이다. 인가 모듈에서 사용할 수 있는 속성 외에도 어드미션 제어 모듈은 생성되거나 수정된 오브젝트 내용에 접근할 수 있다.

어드미션 제어 모듈은 생성되거나 수정된 오브젝트 내용에 접근가능

4. Admission Control

예를 들어 다음 배포 요청이 들어왔다고 해보겠습니다.

yaml

```
spec:
  containers:
    - image: unknown-registry.example/app:latest
      securityContext:
        privileged: true
```

Admission Controller는 이 내용을 보고 조직의 정책과 비교할 수 있습니다.

○ 채팅에 추가

허용되지 않은 Registry인가?	→ 예
latest 태그를 사용했는가?	→ 예
privileged 컨테이너인가?	→ 예

정책을 위반했으므로 요청을 거부할 수 있습니다.

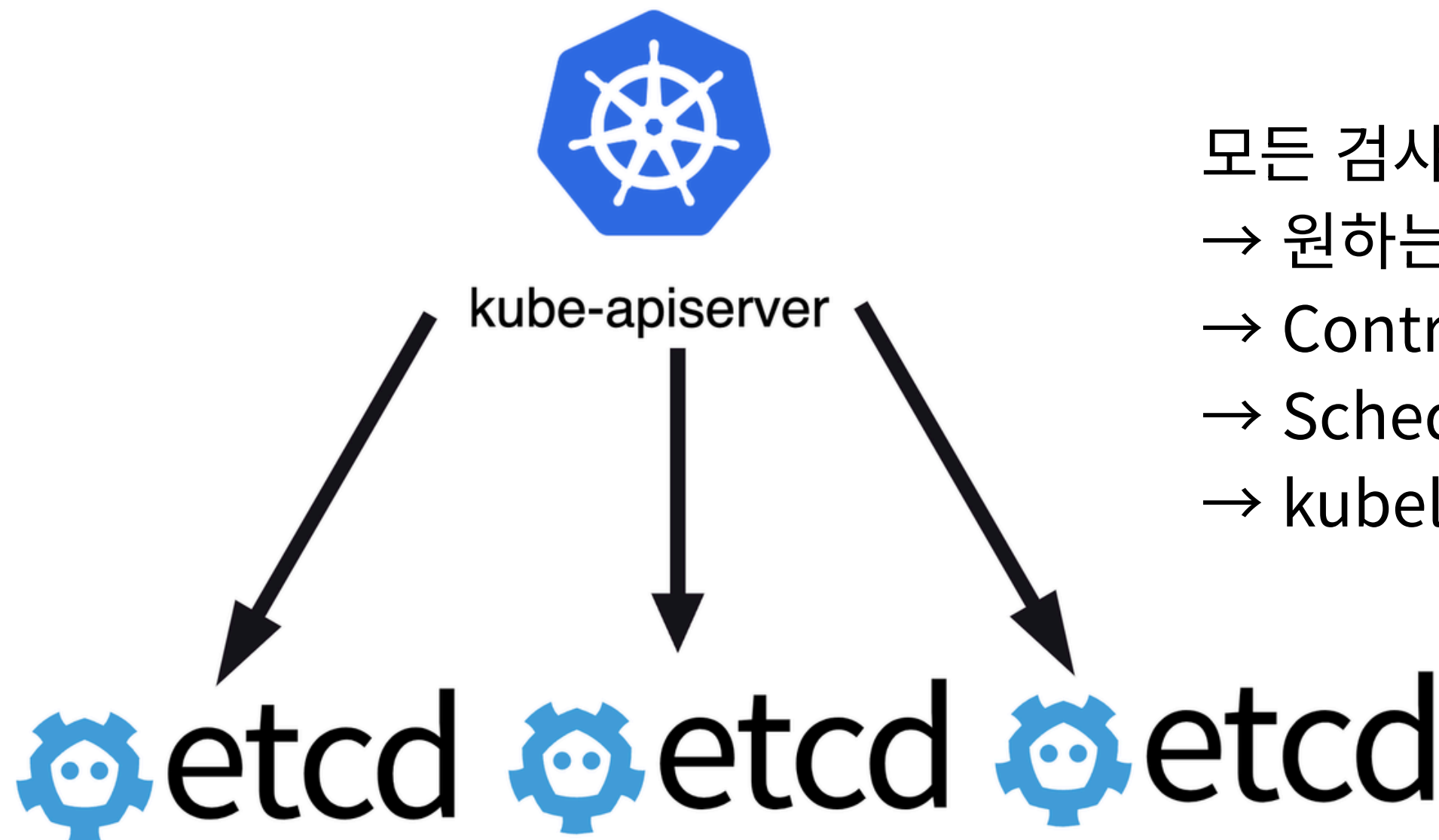
4. Admission Control

구분	질문
인증	누구인가?
인가	이 작업을 할 권한이 있는가?
Admission	작업 내용이 안전한가?

인가와 Admission의 차이

5. 유효성 검사와 저장

모든 보안 관문을 통과하면 요청 형식이 쿠버네티스 API 규격에 맞는지 검사한 후 etcd에 저장



모든 검사 통과

→ 원하는 상태를 etcd에 저장

→ Controller가 필요한 오브젝트 생성

→ Scheduler가 실행할 노드 결정

→ kubelet이 Pod 실행

분산 시스템의 중요 데이터를 안전하게 저장하는 오픈소스 키-값(Key-Value) 저장소

6. 감사(Audit)

누가 언제 어떤 작업을 했는지 기록

누가: ci-deployer

언제: 14:30

무엇을: production Deployment 수정

결과: 허용



ci-deployer가
production 네임스페이스의
my-app Deployment를
update했고
요청 결과는 200 OK였다.

감사는 요청을 직접 차단하는 기능은 아니지만, **공격 탐지와 사고 조사**에 중요

공급망 공격에 대응하면

공격자가 CI/CD 토큰을 훔쳤다고 가정해보겠습니다

탈취한 CI/CD 토큰으로 요청



인증: 정상 CI/CD처럼 보여 통과할 수 있음



인가: 배포 권한이 있어 통과할 수 있음

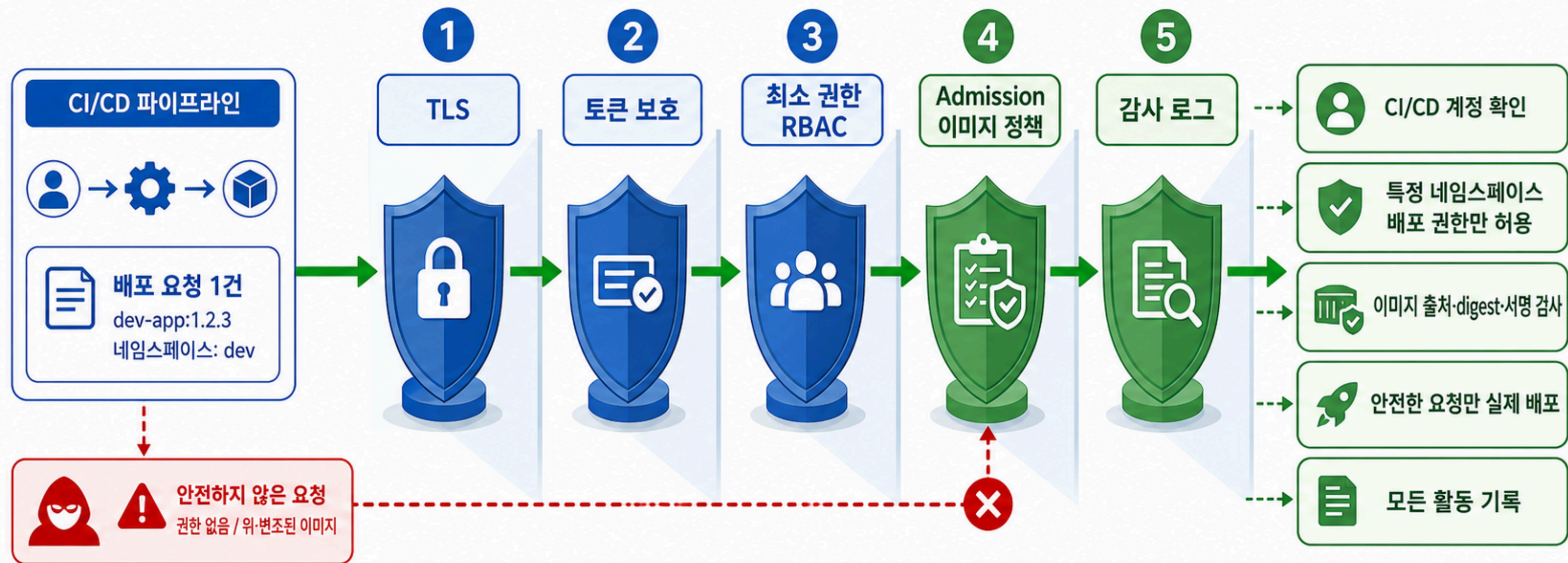


Admission: 서명되지 않은 이미지라서 거부



악성 이미지 배포 차단





인증은 신원을 확인하고, RBAC은 행동 범위를 제한하며, Admission은 위험한 배포 내용을 마지막으로 차단합니다.

 Search this site

- ▶ Documentation
- ▶ Getting started
- ▶ Concepts
- ▶ Tasks
- ▶ Tutorials
- ▼ Reference
 - Glossary
 - ▶ API Overview
 - ▼ API Access Control
 - Authenticating
 - Authenticating with Bootstrap Tokens
 - Authorization
 - Using RBAC Authorization
 - Using Node Authorization
 - Webhook Mode
 - Using ABAC Authorization

[Kubernetes Documentation](#) / [Reference](#) / [API Access Control](#) / Admission Control

Admission Control in Kubernetes

This page provides an overview of *admission controllers*.

An admission controller is a piece of code that intercepts requests to the Kubernetes API server prior to persistence of the resource, but after the request is authenticated and authorized.

Several important features of Kubernetes require an admission controller to be enabled in order to properly support the feature. As a result, a Kubernetes API server that is not properly configured with the right set of admission controllers is an incomplete server that will not support all the features you expect.

What are they?

Admission controllers are code within the Kubernetes API server that check the data arriving in a request to modify a resource.

Admission controllers apply to requests that create, delete, or modify objects. Admission controllers can also block custom verbs, such as a request to connect to a pod via an API server proxy. Admission controllers do *not* (and cannot) block requests to read (**get**, **watch** or **list**) objects, because reads bypass the admission control layer.

Admission control mechanisms may be *validating*, *mutating*, or both. Mutating controllers may modify the data for the resource being modified; validating controllers may not.

The admission controllers in Kubernetes 1.36 consist of the [list](#) below, are compiled into the `kube-apiserver` binary, and may only be configured by the cluster administrator.

-  [Edit this page](#)
-  [Create child page](#)
-  [Create an issue](#)
-  [Print entire section](#)

What are they?

Admission control
extension points

Admission control phases

Why do I need them?

How do I turn on an
admission controller?

How do I turn off an
admission controller?

Which plugins are enabled
by default?

What does each admission
controller do?

AlwaysAdmit

AlwaysDeny

AlwaysPullImages

CertificateApproval

CertificateSigning

CertificateSubjectRestrictio

DefaultIngressClass

DefaultStorageClass

공급망 공격자가 정상 CI/CD 계정을 탈취해 배포 권한을 얻더라도,
위험하거나 승인되지 않은 이미지를 마지막 단계에서 차단하는 것이 Admission Controller의 핵심 역할

Search this site

- ▶ Documentation
- ▶ Getting started
- ▼ Concepts
 - ▶ Overview
 - ▶ Cluster Architecture
 - ▶ Containers
 - ▶ Workloads
 - ▶ Services, Load Balancing, and Networking
 - ▶ Storage
 - ▶ Configuration
- ▼ Security
 - Cloud Native Security
 - Pod Security Standards
 - Pod Security Admission
 - Service Accounts
 - Pod Security

Role Based Access Control Good Practices

Principles and practices for good RBAC design for cluster operators.

Kubernetes RBAC is a key security control to ensure that cluster users and workloads have only the access to resources required to execute their roles. It is important to ensure that, when designing permissions for cluster users, the cluster administrator understands the areas where privilege escalation could occur, to reduce the risk of excessive access leading to security incidents.

The good practices laid out here should be read in conjunction with the general [RBAC documentation](#).

General good practice

Least privilege

Ideally, minimal RBAC rights should be assigned to users and service accounts. Only permissions explicitly required for their operation should be used. While each cluster will be different, some general rules that can be applied are :

- Assign permissions at the namespace level where possible. Use RoleBindings as opposed to ClusterRoleBindings to give users rights only within a specific namespace.
- Avoid providing wildcard permissions when possible, especially to all resources. As Kubernetes is an extensible system, providing wildcard access gives rights not just to all object types that currently exist in the cluster, but also to all object types which are created in the future.
- Administrators should not use `cluster-admin` accounts except where specifically needed.

- [Edit this page](#)
- [Create child page](#)
- [Create an issue](#)
- [Print entire section](#)

General good practice

Least privilege

Minimize distribution of privileged tokens

Hardening

Periodic review

Kubernetes RBAC - privilege escalation risks

Listing secrets

Workload creation

Persistent volume creation

Access to `proxy` subresource of Nodes

Escalate verb

Bind verb

Impersonate verb

CSRs and certificate issuing

Token request

Control admission webhooks

Namespace modification

사용자, Pod, CI/CD에 업무 수행에 꼭 필요한 권한만 주고 나머지는 주지 않는다.

공유하면 좋을 것 같은 것 링크

The screenshot shows a GitHub repository page for `sysnet4admin/_Book_k8sInfra`. The left sidebar displays the file structure, with the `docs/k8s-stnd-arch/2026` directory selected. The main content area shows the commit history and the README.md file content.

Commit History:

Name	Last commit message	Last commit date
..		
img	upt 2026-k8s-stnd-arch	8 months ago
2026-k8s-stnd-arch.pdf	upt 2026-k8s-stnd-arch	8 months ago
README.md	Add English version of README and language toggle links	8 months ago
README_en.md	Add English version of README and language toggle links	8 months ago

README.md Content:

2026년 쿠버네티스 표준 아키텍처

[English Version](#)

표준 아키텍처 선정 배경

2026년 쿠버네티스는 컨테이너를 관리하는 부분에 있어서 경쟁자가 없는 프로젝트가 되었습니다. 그럼에도 2026년에는 매우 큰 변화가 있었는데, OCI의 적극적인 도입과 Ingress NGINX의 지원 종료 예정입니다. 이와 관련한 자세한 내용은 헬름, 하버 그리고 인그레스를 참고하시기 바랍니다. 이와 같이 쿠버네티스는 단일 제품으로 존재하는 것이 아니라 쿠버네티스 상에서 동작하는 다양한 컨테이너 애플리케이션을 유기적으로 연결하여 다양한 목적으로 사용하게 됩니다. 따라서 이를 위한 성숙한 프로젝트들은 매우 중요한 요소입니다. 2025년 10월 28일 을 기준으로 클라우드 네이티브 컴퓨팅 재단 (CNCF, Cloud Native Computing Foundation)의 졸업 프로젝트는 총 34개가 발표되었습니다.

CNCF Graduated Projects (Oct 28, 2025) –
Crossplane, Dragonfly

https://github.com/sysnet4admin/_Book_k8sInfra/tree/main/docs/k8s-stnd-arch/2026

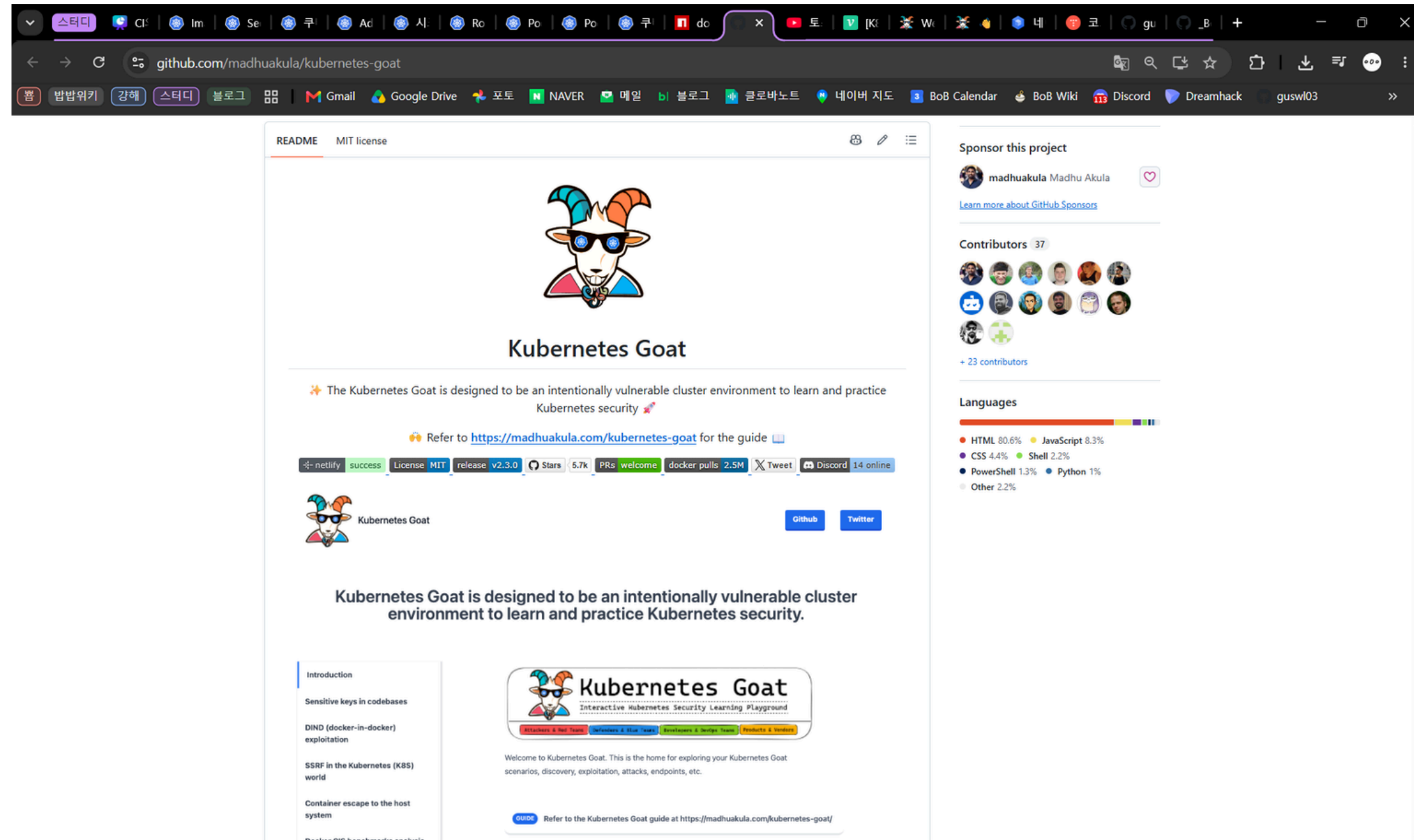
공유하면 좋을 것 같은 것 링크



Introduction | Kubernetes Goat

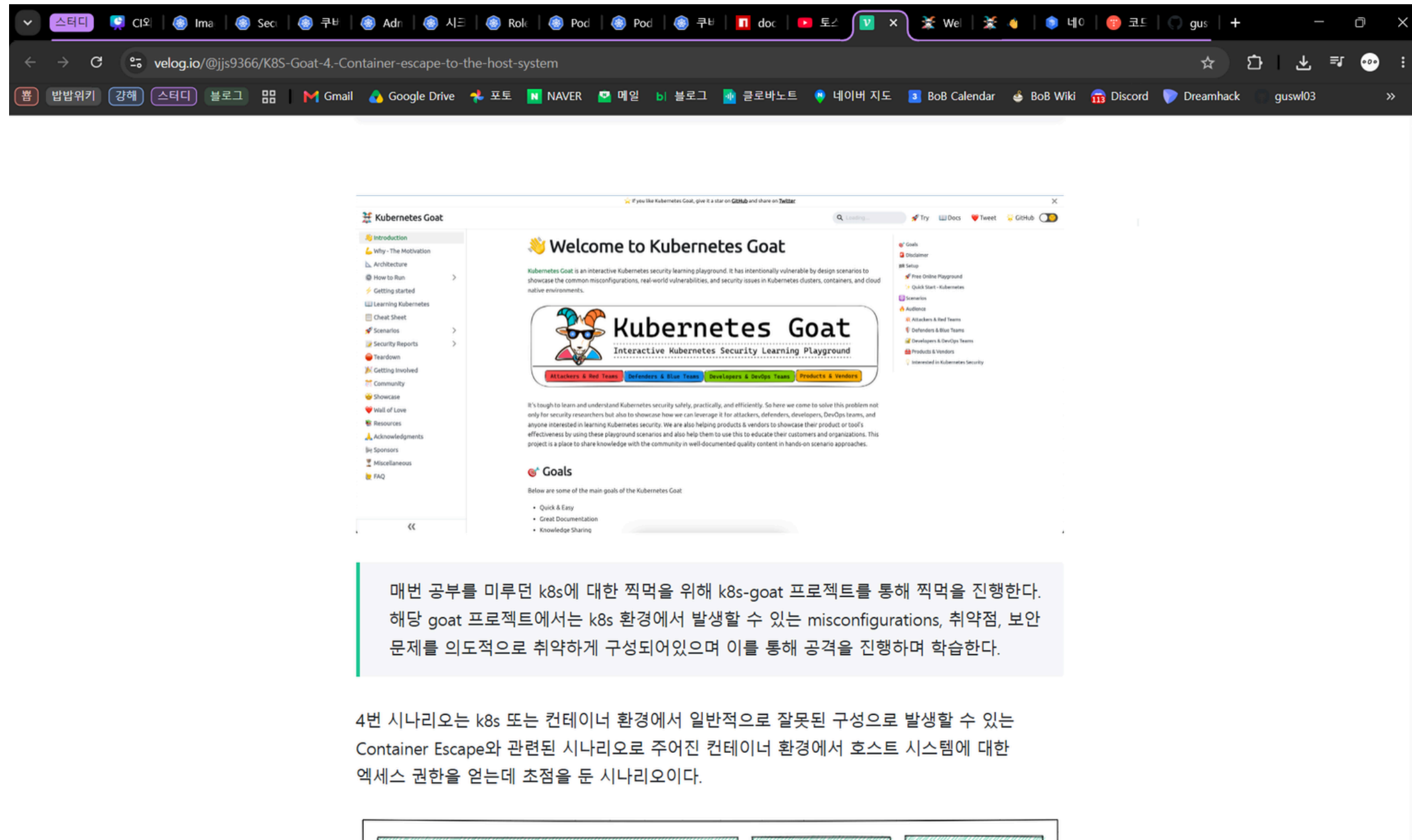
Kubernetes Goat is an interactive Kubernetes security learning playground. It has intentionally vulnerable by design scenarios to showcase the common misconfigurations, real-world vulnerabilities, and security issues in Kubernetes...

공유하면 좋을 것 같은 것 링크



<https://github.com/madhuakula/kubernetes-goat>

공유하면 좋을 것 같은 것 링크



<https://velog.io/@jjs9366/K8S-Goat-4.-Container-escape-to-the-host-system>

공유하면 좋을 것 같은 것 링크

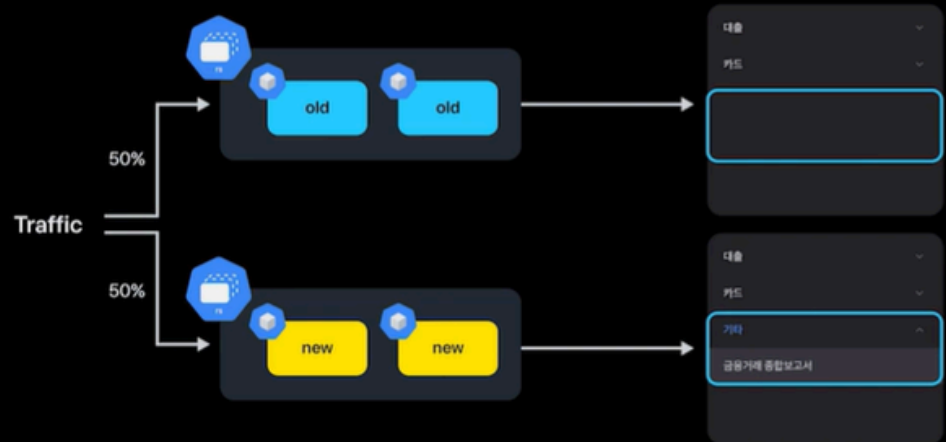
Premium

KR

검색

+ 만들기

Canary 배포와 404에러



SLASH 24



토스 | SLASH 24 - 생산성과 안정성 모두 잡는 마스터키, Canary 배포 개선기

토스

구독자 15.2만명

구독

91

공유

질문하기

저장

조회수 3.5천회 1년 전 #SLASH #canary #sre

생산성을 해치지 않으면서, 배포도 안전하게 할 수는 없을까요?

이 동영상에 대해 물어보세요

이 영상은 토스뱅크에서 서비스 안정성과 개발 생산성을 동시에 확보하기 위해 도입한 **카나리(Canary)** 배포 개선 사례를 다루고 있습니다. 주요 내용은 다음과 같습니다.

1. 배경 및 기존 배포 전략 (0:00 - 3:31)

- 롤링 업데이트의 한계(배포 중단 불가, 롤백 시 긴 복구 시간)를 설명하며, 트래픽을 점진적으로 전환하여 장애 대응이 용이한 **카나리 배포**의 장점을 강조합니다.

2. 프론트엔드 카나리 배포의 문제점과 해결책: 스티키 카나리 (3:32 - 14:24)

- **문제점:** 카나리 배포 중 유저가 구버전과 신버전을 오가며 기능 오류(페이지 Not Found 등)를 겪는 버전 스위칭 문제.
- **해결책 (스티키 카나리):** 유저의 고유 키를 기반으로 해싱(Hashing)하여 한번 할당된 버전이 배포 완료 시까지 고정되도록 함으로써 일관된 사용자 경험을 제공.

3. 자동 장애 대응: 오토 카나리(Auto-Canary) (14:25 - 20:30)

- **문제점:** 사람이 수동으로 모니터링하는 것의 한계 및 MSA 환경에서 발생하는 복잡한 장애 전파.
- **해결책:** 복잡한 오픈소스 대신, 자체 시스템(비바 시스템) 내에서 에러 로그 수와 로그 지연(Lag)을 실시간 체크하여 임계치 초과 시 **자동으로 롤백**하는 간단하고 효과적인 메커니즘을 도입.

결론: 복잡한 외부 도구 도입보다는 서비스 상황에 맞는 ****가장 간단한 방법 (MVP)****으로 문제를 해결하는 것이 장애 시간을 최소화하는 핵심임을 강조합니다.

스티키 카나리의 핵심 원리는?

질문하기...

Gemini에게 물어보기

모두

시리즈

토스 제공

감상한 동영상

dingo

FTISLAND(FT아일랜드)의 킬링 부이스를 라이브로! - 사랑앞이

<https://www.youtube.com/watch?v=ApNj9MZU7Ak&t=323s>

Slopsquatting(슬롭스쿼팅)

**LLM이 존재하지 않는 패키지 이름을 추천하고,
공격자가 그 이름으로 악성 패키지를 먼저 등록하여
사용자의 설치를 유도하는 공급망 공격입니다.**

<https://ggonmerr.tistory.com/595>

Slopsquatting(슬롭스쿼팅)

개발자가 LLM에 코드 생성 요청



LLM이 존재하지 않는 패키지 추천



공격자가 해당 이름을 npm·PyPI 등에 등록



다른 개발자가 같은 추천을 받음

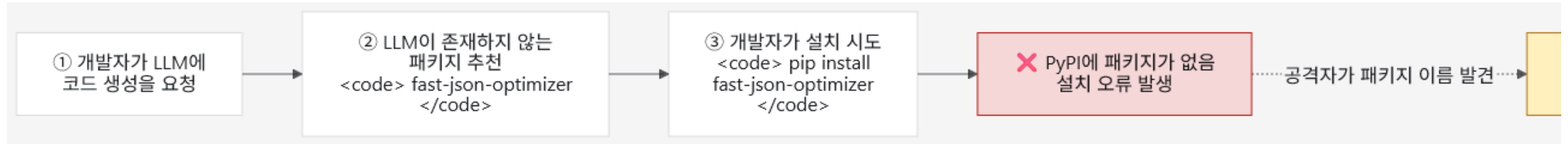


npm install 또는 pip install 실행



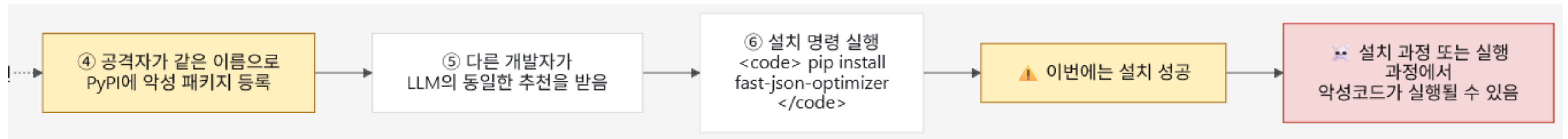
악성 설치 스크립트·코드 실행

예를 들면 LLM이 다음처럼 답한다고 가정해봅시다.



`pip install fast-json-optimizer`

그런데 `fast-json-optimizer`가 실제로 존재하지 않는 패키지였다면 처음에는 설치 오류만 발생합니다.



공격자가 이 이름을 발견해서 PyPI에 악성 패키지를 등록하면 그다음부터 설치 명령이 정상적으로 성공하면서 악성코드가 실행될 수 있습니다.

We Have a Package for You! – USENIX Security 2025
패키지 환각과 Slopsquatting의 기준 연구입니다.

<https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>

Importing Phantoms - 2025

모델 · 언어 · 프롬프트별 차이와 평가 지표를 볼 수 있습니다.



Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities

Large Language Models (LLMs) have become an essential tool in the programmer's toolkit, but their tendency to hallucinate code can be used by malicious actors to introduce vulnerabilities to broad...

Hallucination by Code Generation LLMs: Taxonomy, Benchmarks, Mitigation, and Challenges – 2025

패키지 환각을 포함한 코드 생성 환각 전체를 정리한 조사 논문입니다.



Hallucination by Code Generation LLMs: Taxonomy, Benchmarks,...

Recent technical breakthroughs in large language models (LLMs) have enabled them to fluently generate source code. Software developers often leverage both general-purpose and code-specialized LLMs...

PackMonitor – 2026

생성 과정에서 허용된 패키지 이름만 나오게 제한하는 최신 방어 방향입니다.



PackMonitor: Enabling Zero Package Hallucinations Through...

As Large Language Models (LLMs) are increasingly integrated into software development workflows, their trustworthiness has become a critical concern. However, in dependency recommendation...

BOUND – 2026

모델 편집을 이용한 패키지 환각 완화 연구입니다.



Mitigating Package Hallucinations in Large Language Models via...

Large language models (LLMs) have demonstrated strong capabilities in software engineering tasks, such as code generation, library recommendation, and dependency configuration. However, recent...



왜 ChatGPT는 우리를 해킹했을까?

조회수 1.2만회 · 19시간 전



미쳐 날뛰어버린 GPT 이야기. 세상 밖으로 뛰어나와 해킹을
한 사건

조회수 2.3만회 · 3일 전

